



دانشکده‌گان علوم و فناوری‌های میان‌رشته‌ای

دانشکده سامانه‌های هوشمند

گروه علوم و فناوری داده

درس کاربرد سیستم‌های هوشمند در پزشکی

استاد: دکتر مهدی تیموری

گردآورنده: مهدیس باقری دورفرد

INDEX

Imports.....	3
Load and prepare data.....	3
Answer to questions: : Part one	4
Clause based CSP with BT-FC.....	6
Version 1 : Using deepcopy	6
Version 2: Using Trail Stack	16
Final Version: Time and complexity check + version 2	18
Answer to questions : Part two	19
Walksat	22
Answer to questions : Part three.....	27
Analysis and Solutions	28
Conclusion	30

```
from scipy.io import loadmat
```

برای خواندن فایل های `.mat`. دیتاست های مسائل SAT یا CSP در متلب ساخته شده اند و تبدیل آن ها به ساختارهای قابل استفاده در پایتون.

```
import numpy as np
```

برای مدیریت و پردازش کارآمد داده های عددی و آرایه ای

```
from copy import deepcopy
```

برای کپی کامل دامنه ها در نسخه ی اولیه ی Forward Checking به منظور جلوگیری از تداخل تغییرات در مراحل مختلف backtracking.

```
from itertools import product
```

برای تولید تمام ترکیب های ممکن True/False در دامنه

```
import time
```

برای اندازه گیری زمان اجرای الگوریتم

Load and prepare data

```
data_20 = loadmat("SAT_20Var.mat")
```

به کمک `loadmat`، فایل `.mat` لود میشود و در متغیر `data_20` ذخیره میشود.

```
p_20 = data_20['P']
```

`data_20` یک دیکشنری است. با اینکار، `value` مرتبط با کلید 'P' را در `p_20` میریزیم.

```
unique_values_20 = np.unique(p_20)
count_unique_20 = len(unique_values_20)
```

```
print(count_unique_20)
print(unique_values_20)
```

```
40
[-20 -19 -18 -17 -16 -15 -14 -13 -12 -11 -10 -9 -8 -7 -6 -5 -4 -3
 -2 -1  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
 17 18 19 20]
```

صرفاً جهت آشنایی بیشتر با دیتا، بررسی شده که در `p_20` چند مقدار یکتا وجود دارد. با توجه به خروجی، واضح هست که ۴۰ مقدار یکتا وجود دارد که شامل ۲۰ لیترال و `not` آن هاست (همه لیترال ها حداقل یکبار به صورت ساده و `not` شده وجود دارند). این مراحل برای دو فایل `.mat` دیگر نیز تکرار میشود.

الف) قیود را به دقت توصیف کنید و مشخص نمایید چگونه باید Forward Checking را انجام داد.

مدلسازی مسئله 3SAT به صورت CSP

برای حل مسئله 3SAT با استفاده از Backtracking، مسئله به صورت یک مسئله ارضای قیود (CSP) مدل می‌شود:

- متغیرها (Variables):

هر گزاره (Clause) یک متغیر در CSP در نظر گرفته می‌شود.

- دامنه‌ها (Domains):

دامنه‌ی هر گزاره شامل تمام حالت‌های ممکن True/False برای سه متغیر آن گزاره است که مقدار گزاره را True می‌کنند.

چون از ۸ حالت ممکن برای سه متغیر فقط حالت (False, False, False) نامعتبر است، بنابراین $\text{Domain}=7$

- تعریف قیود (Constraints)

قیود بین گزاره‌ها به دلیل اشتراک متغیرها ایجاد می‌شوند. اگر دو گزاره دارای یک متغیر مشترک باشند، مقدار اختصاص داده شده به آن متغیر در

هر دو گزاره باید یکسان باشد. بنابراین قید به این صورت تعریف می‌شود:

برای هر دو گزاره‌ی همسایه، مقادیر مشترک متغیرهای منطقی آن‌ها نباید با هم در تناقض باشند.

- الگوریتم Backtracking

الگوریتم به صورت زیر عمل می‌کند:

۱. در ابتدا هیچ گزاره‌ای مقداردهی نشده است.

۲. در هر مرحله:

یک گزاره انتخاب می‌شود.

یکی از حالات مجاز دامنه‌ی آن به عنوان مقدار انتخاب می‌شود.

۳. اگر تناقضی ایجاد نشود، الگوریتم به مرحله‌ی بعد می‌رود.

۴. در صورت بروز بن‌بست، Backtracking انجام می‌شود.

- Forward Checking (پخش قیود)

پس از مقداردهی به یک گزاره، از Forward Checking برای کاهش فضای جست‌وجو استفاده می‌شود:

مراحل Forward Checking:

۱. گزاره‌ی مقداردهی شده به عنوان آخرین متغیر انتخاب شده در نظر گرفته می‌شود.

۲. تمام گزاره‌های همسایه (یعنی گزاره‌هایی که متغیر مشترک دارند) بررسی می‌شوند.

۳. برای هر همسایه:

تمام حالت‌های دامنه‌ی آن بررسی می‌شود.

هر حالتی که با مقدار گزاره‌ی فعلی در متغیرهای مشترک ناسازگار باشد، حذف می‌شود.

۴. اگر دامنه‌ی یکی از همسایه‌ها خالی شود:

شکست اعلام می‌شود.

Backtracking انجام می‌گیرد.

نتیجه‌ی Forward Checking: 🌈

حذف زودهنگام حالت‌های نامعتبر

جلوگیری از ورود به شاخه‌های بی‌ثمر جست‌وجو

- مرتب‌سازی گزاره‌ها بر اساس اندازه دامنه (MRV)

برای افزایش کارایی الگوریتم، از هیوریستیک Minimum Remaining Values (MRV) استفاده می‌شود

در هر مرحله:

گزاره‌ای انتخاب می‌شود که کمترین اندازه‌ی دامنه‌ی مجاز را دارد.

دلیل:

گزاره‌هایی با دامنه کوچک‌تر محدودتر هستند.

مقداردهی زودهنگام آن‌ها احتمال شکست زودرس و کاهش Backtracking را افزایش می‌دهد.

- جمع‌بندی

در این روش:

قیود از طریق اشتراک متغیرهای منطقی بین گزاره‌ها تعریف می‌شوند.

Forward Checking با حذف مقادیر ناسازگار از دامنه‌ی گزاره‌های آینده، فضای جست‌وجو را کاهش می‌دهد.

مرتب‌سازی گزاره‌ها بر اساس اندازه دامنه (MRV) باعث افزایش سرعت همگرایی الگوریتم می‌شود.

ترکیب Backtracking + Forward Checking + MRV یک روش مؤثر برای حل مسئله 3sat با ابعاد بزرگ است.

تابع build_domain

این قسمت از کد مربوط به ساخت Domain برای هر Clause در مدل Clause-Based CSP برای مسئله است.

در روش Clause-Based CSP:

- هر Clause یک متغیر در CSP محسوب می‌شود.
- Domain آن Clause شامل همه حالت‌های انتسابی است که Clause را True می‌کنند.
- چون هر Clause در 3SAT سه literal دارد، کل حالت‌های ممکن 23 حالت (8 حالت) است.
- فقط 7 حالت مجاز هستند چون یکی از حالت‌ها clause را False می‌کند.
- پس Domain هر Clause $v = \text{assignment}$ معتبر.

```
def build_domains(clauses):
```

- ورودی تابع clause ها هستند.
- هر عدد مثبت یک متغیر معمولی و هر عدد منفی نقیض متغیر است. مثال:

$$[1, -2, 3] = (x1 \vee \neg x2 \vee x3)$$

```
domains = []
```

```
domains = [
    [state1, state2, ...], # clause 1
    [state1, state2, ...], # clause 2
]
```

- در ابتدا لیست domains خالی است.
- در پایان domains[i]، domain مربوط به clause i است. مثال:

```
for clause in clauses:
```

- در این حلقه، همه ی clause ها به ترتیب انتخاب میشوند

```
possible_states = []
```

- در این لیست، 7 حالت معتبر clause ذخیره می‌شود.

```
for values in product([False, True], repeat=3):
```

- تابع product از itertools، ۸ حالت ممکن برای سه متغیر تولید میکند. حالت ها بدین صورت هستند:

```
(False, False, False)
(False, False, True)
(False, True, False)
(False, True, True)
(True, False, False)
(True, False, True)
(True, True, False)
(True, True, True)
```

```
satisfied = False
state = {}
```

- متغیر satisfied برای بررسی true شدن گزاره استفاده میشود.

```
for lit, val in zip(clause, values):
```

- در این حلقه، literal ها بررسی میشوند. به ازای هر literal و یک value (از clause انتخاب شده در حلقه قبلی) این حلقه اجرا میشود مثال: برای این کلاز و این مقدار داریم:

```
clause = [1, -2, 3]
values = (True, False, True)
```

```
(1 , True)
(-2, False)
(3 , True)
```

```
var = abs(lit)
state[var] = val
```

متغیر به همراه مقدارش در دیکشنری state ذخیره میشود. برای مثال x1: True

```
literal_value = val if lit > 0 else not val
```

- مهم ترین بخش تابع:

اگر literal مثبت است، literal_value = val و اگر نه literal_value = not val

مثال: اگر val = True lit = 1 آنگاه literal_value = True و اگر val = True lit = -2 آنگاه literal_value = False

```
if literal_value:
    satisfied = True
```

- اگر literal_value true شده باشد، satisfied که در واقع یک flag است، true میشود. یعنی اگر حداقل یکی از literal ها True شود، clause satisfied است. این دقیقاً همان تعریف منطقی OR در clause است: (1|1|2|3)

```
if satisfied:    exclude all-false case
    possible_states.append(state)
```

- اگر satisfied true شده باشد، آن state به لیست possible_state افزوده میشود.
- سپس الگوریتم به ابتدای حلقه ی for اول برمیگردد تا clause بعدی بررسی شود.
- پس از بررسی آخرین clause، لیست domains، return میشود.

🚩 تابع build_neighbors

این تابع ساختار همسایگی را می‌سازد. این ساختار بعداً در Forward Checking استفاده می‌شود تا بدانیم کدام clause ها روی هم اثر می‌گذارند.

نکته: بین دو clause ، constraint وجود دارد اگر یک متغیر مشترک داشته باشند. مثلاً اگر هر دو شامل x3 باشند. تابع build_neighbors دقیقاً همین ارتباطها را پیدا می‌کند.

```
def build_neighbors(clauses):
```

- ورودی تابع clause ها هستند.

```
n = len(clauses)
```

- تعداد clause ها در متغیر n ذخیره میشوند.

```
neighbors = {i: set() for i in range(n)}
```

```
neighbors = {
    0: set(),
    1: set(),
    2: set()
}
```

- اینجا برای هر clause یک مجموعه خالی از همسایه‌ها ساخته می‌شود و در دیکشنری neighbors ذخیره میشود. پس در ابتدا داریم:
- استفاده از set برای این است که: تکراری اضافه نشود و جستجو سریع باشد.

```
for i in range(n):
    vars_i = set(abs(l) for l in clauses[i])
```

حلقه ی for به ازای همه ی clause ها اجرا میشود و متغیرهای هر clause را استخراج میکند و در vars_i میریزد.

مثال: اگر داشته باشیم $clause[0] = [1, -2, 3]$ آنگاه $vars_i = \{1, 2, 3\}$

```
for j in range(i + 1, n):
```

این حلقه ی for، برای بررسی clause های بعدیست. چرا $i+1$ ؟ چون: هر جفت فقط یکبار بررسی شود و مقایسه با خودش انجام نشود.

```
vars_j = set(abs(l) for l in clauses[j])
```

استخراج متغیرهای clause دوم

```
if vars_i & vars_j: intersection not empty
    neighbors[i].add(j)
    neighbors[j].add(i)
```

بررسی اشتراک متغیر در دو clause مورد بررسی. & یعنی intersection در set. اگر دو clause اشتراک داشته باشند، همسایه هستند و در دیکشنری neighbors، به عنوان همسایه های یکدیگر، ثبت میشوند.

بعد از اتمام حلقه دوم، تمام clause های بعدی بررسی شده و تابع به حلقه اول برمیگردد و clause بعدی برای بررسی انتخاب میشود. در نهایت return neighbors، میشود.

تابع consistent

وقتی دو clause روی یک مقدار یک متغیر، اشتراک نظر دارند، باهم سازگارند.

در Backtracking + Forward Checking الگوریتم با این تابع چک می‌کند:

- وقتی برای یک clause مقدار انتخاب می‌کنی (state_i)
- باید همه domain‌های clause‌های همسایه را بررسی کنی (neighbors[i])
- و از آنها state‌هایی را که با این state ناسازگارند حذف کنی.

یعنی این تابع ابزار اصلی تشخیص ناسازگاری در مرحله‌ی Forward Checking است

```
def consistent(state1, state2):
```

- تابع، دو state دریافت می‌کند. هر state یک دیکشنری است. برای مثال:

```
state1 = {1: True, 2: False, 3: True}
state2 = {2: False, 3: True, 4: True}
```

```
for var in state1:
    if var in state2:
```

- حلقه‌ی for برای پیمایش روی هر متغیر در state1 و شرط if برای پیدا کردن متغیرهای مشترک. (چک می‌کند آیا همان متغیر در state2 هم وجود دارد یا نه) در مثال بالا: متغیرهای مشترک = {۲, ۳}

```
    if state1[var] != state2[var]:
        return False
```

- بررسی ناسازگاری مقداری: یعنی اگر متغیر مشترک در یکی True و دیگری False باشد، ناسازگار هستند و False را return می‌شود.

```
    return True
```

فقط اگر هیچ تضادی پیدا نشود، به معنی consistency است.

بنابراین خروجی تابع consistent، مقادیر True/False می‌باشد.

تابع forward_check

این تابع مسئول کاهش فضای جستجو است تا از رفتن به مسیرهای بی‌فایده جلوگیری کند. وقتی در الگوریتم Backtracking، یک assignment قطعی (value) برای یک متغیر (clause) انجام می‌شود:

- forward_check باید دامنه (domain) clause‌های همسایه آن را بررسی کند.
- تمام assignment‌های موجود در domain همسایه‌ها که با assignment جدید ناسازگار هستند را حذف می‌کند.
- اگر domain هر همسایه تهی شود، یعنی با این assignment فعلی، دیگر هیچ راه حلی وجود ندارد، پس باید بلافاصله Fail کند.

```
def forward_check(var_idx, assignment, domains, neighbors, trail):
```

ورودی‌ها:

- var_index: ایندکس clause‌ای که تازه برایش یک assignment انجام شده (مثلاً 0 clause).
- assignment: دیکشنری فعلی از assignment‌های انجام شده
- domains: لیست فعلی domain‌ها برای تمام clause‌ها. این لیست در حین اجرای الگوریتم تغییر می‌کند (کاهش می‌یابد).
- neighbors: ساختار همسایگی که قبلاً توضیح داده شد.

```
new_domains = deepcopy(domains)
```

- اگر ما مستقیم روی domains اصلی تغییر ایجاد کنیم و بعداً متوجه شویم این مسیر به جواب نمی‌رسد (Backtrack کنیم)، دامنه‌های اصلی خراب شده‌اند. با deepcopy یک نسخه‌ی کاملاً مستقل و «ایزوله» می‌سازیم تا تغییرات ما فقط روی همین شاخه از درخت جستجو اعمال شود.

```
for neighbor in neighbors[var_index]:
```

- این حلقه روی تمام clause‌هایی که همسایه var_index هستند، پیمایش می‌کند.
- فقط همسایه‌ها، چون فقط آنها می‌توانند با var_index متغیر مشترک داشته باشند و constraint ایجاد کنند.

```
if neighbor in assignment:  
    continue
```

- اگر clause همسایه (neighbor) خودش از قبل assign شده باشد (یعنی یک مقدار قطعی گرفته باشد)، دیگر نیازی نیست domain آن را چک کنیم. چون Assignment آن قبلاً انجام شده و consistency آن با var_index باید در مراحل قبلی چک شده باشد.

```

filtered_domain = []

for state in new_domains[neighbor]:
    if consistent(assignment[var_index], state):
        filtered_domain.append(state)

```

- برای هر state که در domain فعلی neighbor وجود دارد :
چک می کند که آیا state با assignment فعلی سازگار است یا نه (با استفاده از تابع consistent که قبلاً توضیح داده شد). اگر سازگار بود، آن state را به filtered_domain اضافه می کند.

```

if not filtered_domain:    domain wiped out
    return False, None

```

- بررسی تهی شدن domain همسایه انتخابی
اگر بعد از بررسی تمام state ها، filtered_domain تهی باقی بماند :
یعنی هیچ state ای در domain neighbor وجود ندارد که با assignment[var_index] سازگار باشد، این یعنی مسیر فعلی به جواب نمی رسد. پس forward_check بلافاصله False برمی گرداند .

```

new_domains[neighbor] = filtered_domain

```

- به روزرسانی domain همسایه
اگر filtered_domain تهی نبود، یعنی هنوز امید هست و فقط دامنه ی همسایه به روز میشود.

```

return True, new_domains

```

- اگر forward_check تمام همسایه ها را بررسی کرد و هیچ کدام از domain ها تهی نشدند، یعنی این assignment فعلی هنوز قابل ادامه دادن است . پس forward_check مقدار True را به همراه new_domains برمی گرداند

نتیجه : forward_check با حذف انتخاب های ناسازگار در آینده، branching بیهوده در درخت جستجو را به شدت کاهش می دهد و الگوریتم را بسیار کارآمدتر می کند. این کار را قبل از موعد انجام می دهد تا از بستن های آینده زودتر مطلع شود.

تابع select_unassigned_variable

هیوریستیک MRV متغیری را برای مقداردهی انتخاب می کند که کمترین تعداد مقادیر ممکن در دامنه ی خود را داشته باشد. منطق این روش این است که اگر متغیری گزینه های بسیار محدودی داشته باشد، بهتر است زودتر بررسی شود. در این صورت اگر آن متغیر باعث ایجاد بن بست شود، این بن بست سریع تر آشکار شده و از جستجوی غیرضروری در شاخه های بزرگ تر جلوگیری می شود.

به بیان دیگر، MRV به الگوریتم کمک می کند تا شکست ها را زودتر کشف کند (Fail-First Principle) و در نتیجه فضای جستجو کوچک تر شود.

```
def select_unassigned_variable(assignment, domains):
    unassigned = [i for i in range(len(domains)) if i not in assignment]
```

- unassigned فهرستی از اندیس Clause هایی است که هنوز مقداردهی نشده اند.

```
return min(unassigned, key=lambda i: len(domains[i]))
```

- Clause ی انتخاب می شود که کمترین تعداد مقدار ممکن در دامنه ی خود داشته باشد
- تابع min روی لیست Clause های مقداردهی نشده اجرا می شود .
- معیار مقایسه (key) برابر با اندازه ی دامنه ی هر Clause است .

(key=lambda i: len(domains[i])) این قسمت مشخص می کند که معیار مقایسه برای تابع min چه باشد

- در نتیجه Clause ی برگردانده می شود که کمترین تعداد state باقی مانده در دامنه ی خود را دارد.

```
def backtrack(assignment, domains, neighbors):
    if len(assignment) == len(domains):
        return assignment
```

- در اینجا بررسی می‌شود آیا برای تمام متغیرهای مسئله مقدار تعیین شده است یا نه.
- بنابراین اگر تعداد clause‌های مقداردهی شده = کل clause‌ها، یک راه‌حل کامل و سازگار پیدا شده و همان assignment برگردانده می‌شود.

```
var = select_unassigned_variable(assignment, domains)
```

- اگر هنوز متغیری بدون مقدار مانده باشد، از MRV heuristic (Minimum Remaining Values) برای انتخاب متغیر بعدی (var) استفاده می‌شود.

```
for state in domains[var]:
```

- برای clause انتخاب‌شده (var)، تمام state‌های ممکن در دامنه بررسی می‌شوند.

```
new_assignment = assignment.copy()
new_assignment[var] = state
```

- ساخت assignment جدید. اینجا:

- assignment قبلی کپی می‌شود
- مقدار جدید برای clause انتخابی اضافه می‌شود

```
ok, new_domains = forward_check(var, new_assignment, domains, neighbors)
```

- اینجا مهم‌ترین pruning انجام می‌شود. Forward Checking بررسی می‌کند: آیا انتخاب فعلی باعث می‌شود دامنه‌ی همسایه‌ها خالی شود یا نه. مراحل:

- (۱) همسایه‌های clause فعلی پیدا می‌شوند
- (۲) دامنه‌ی آنها فیلتر می‌شود
- (۳) state‌های ناسازگار حذف می‌شوند

```
if ok:
    result = backtrack(new_assignment, new_domains, neighbors)
    if result is not None:
        return result
```

- ادامه جستجو به صورت بازگشتی

اگر forward checking موفق باشد، در اینجا الگوریتم ، assignment جدید و دامنه‌های جدید (که prune شده‌اند) را به مرحله بعدی recursion می‌فرستد. در واقع درخت جستجو گسترش پیدا می‌کند.

```
if result is not None:  
    return result
```

- اگر در recursion پایان تر:

یک جواب کامل پیدا شود، آن جواب به بالا propagate می‌شود و کل الگوریتم متوقف می‌شود.

```
return None
```

- اگر هیچ state ای جواب ندهد، حلقه تمام می‌شود.

```
def solve_3sat_csp(matrix):
```

این تابع در واقع تابع اصلی برای اجرای حل کننده ی SAT بر پایه ی CSP است. کار آن این است که داده ی ورودی را آماده کند، ساختارهای لازم برای الگوریتم را بسازد و سپس الگوریتم Backtracking Search را اجرا کند.

ورودی تابع: که یک numpy matrix است .

```
clauses = matrix.tolist()
```

تبدیل numpy به لیست پایتون. چون کار با لیست های پایتون در توابع بعدی ساده تر است.

```
domains = build_domains(clauses)
```

ساخت دامنه ها

```
neighbors = build_neighbors(clauses)
```

ساخت گراف همسایگی

```
solution = backtrack({}, domains, neighbors)
```

اجرای الگوریتم Backtracking

الگوریتم backtrack سپس:

- یک clause انتخاب می کند (MRV)
- یک state از دامنه آن انتخاب می کند
- forward checking انجام می دهد
- recursion ادامه می دهد
- اگر بن بست شود ← backtrack

🚩 تابع `build_domain` : مطابق با ورژن ۱

🚩 تابع `build_neighbors` : مطابق با ورژن ۱

🚩 تابع `consistent` : مطابق با ورژن ۱

🚩 تابع `forward_check`

در این نسخه از الگوریتم Forward Checking همان هدف نسخه‌ی قبلی دنبال می‌شود: بعد از مقداردهی یک متغیر، دامنه‌ی همسایه‌های آن بررسی و مقادیر ناسازگار حذف می‌شوند تا در صورت امکان زودتر ناسازگاری‌ها تشخیص داده شوند. تفاوت اصلی این نسخه با نسخه‌ی قبلی در مدیریت دامنه‌ها (domains) است.

در نسخه‌ی اول برای جلوگیری از تغییر دامنه‌ی اصلی از `deep copy` استفاده می‌شد، اما در این نسخه دامنه‌ها مستقیماً تغییر داده می‌شوند (in-place) و تغییرات در ساختاری به نام `trail` ثبت می‌شوند تا در صورت نیاز بتوان آن‌ها را بازگرداند.

تفاوت‌های اصلی

- حذف Deep Copy:
- این کار باعث کاهش مصرف حافظه و افزایش سرعت می‌شود، زیرا در هر فراخوانی نیازی به کپی کامل ساختار دامنه‌ها نیست.
- اضافه شدن پارامتر `trail`

```
def forward_check(var_idx, assignment, domains, neighbors, trail):
```

پارامتر جدید `trail` برای ثبت تغییراتی است که روی دامنه‌ها اعمال می‌شوند. `trail` یک لیست است که شامل جفت‌های (variable, old_domain) می‌شود. این اطلاعات در زمان `backtracking` استفاده می‌شوند تا دامنه‌ها به حالت قبلی بازگردند.

```
if new_domain != old_domain:
    trail.append((nb, old_domain))
    domains[nb] = new_domain
```

- ثبت تغییر دامنه در Trail

در اینجا اگر دامنه تغییر کرده باشد:

۱. دامنه‌ی قبلی در `trail` ذخیره می‌شود.
۲. دامنه‌ی جدید جایگزین دامنه‌ی قبلی می‌شود.

این کار امکان `undo` کردن تغییرات هنگام `backtracking` را فراهم می‌کند.

تابع undo (Rollback)

```
def undo(trail, domains):
```

ورودی‌ها:

- trail: لیستی از تغییرات دامنه که هنگام Forward Checking ثبت شده‌اند.
- domains: ساختار فعلی دامنه‌ها که درجا تغییر کرده‌اند و باید به وضعیت قبلی بازگردند.

```
for var, old_domain in reversed(trail):
```

پیمایش trail به صورت معکوس انجام میشود. اگر در Forward Checking دامنه‌ها را به ترتیب A، سپس B، سپس C تغییر داده‌ایم، اینجا به ترتیب C، سپس B، سپس A برگردانده می‌شوند. این الگو برای درست برگشتن حالت‌ها در سناریوهایی که وابستگی ترتیبی وجود دارد ضروری است.

```
domains[var] = old_domain
```

دامنه متغیر var را دقیقاً به همان لیستی که قبل از تغییر داشت برمی‌گرداند. چون در trail مرجع دامنه قبلی ذخیره شده، این عملیات یک assign ساده است و هزینه کپی‌برداری ندارد.

تابع select_next_variable: مطابق با ورژن ۱

تابع backtrack

تفاوت اصلی دو نسخه در نحوه مدیریت assignment و domains در طول جست‌وجو است. نسخه اول مبتنی بر copy کردن ساختارها است، در حالی که نسخه دوم از تغییر درجا (in-place) و مکانیزم rollback با trail استفاده می‌کند.

```
assignment[var] = state
```

در اینجا مقداردهی مستقیماً روی assignment اصلی انجام می‌شود. بنابراین برای جلوگیری از باقی ماندن این مقدار هنگام backtracking باید بعداً آن را حذف کنیم. به همین دلیل در انتهای حلقه داریم:

```
del assignment[var]
```

```
undo(trail, domains)
```

این تابع تمام تغییراتی که در Forward Checking روی دامنه‌ها ایجاد شده‌اند را برمی‌گرداند.

تابع solve_3sat_csp: مطابق با ورژن ۱

مقدار دهی اولیه مجموعه counters

به کمک این شمارنده ها، پیچیدگی زمانی و پیچیدگی فضایی را محاسبه میکنیم.

تابع update_domain_space

این تابع، یک تابع کمکی برای ثبت و تحلیل میزان فضای جست و جو در طول اجرای الگوریتم است. این تابع میزان فضای فعلی دامنه ها را حساب می کند و آن را با بیشترین مقداری که تا به حال دیده شده مقایسه می کند و بزرگ ترین مقدار را در شمارنده ی max_total_domain_size ذخیره می کند. به زبان ساده: اندازه ی فعلی تمام دامنه ها را جمع می زند، اگر این عدد بزرگ تر از رکورد قبلی باشد، رکورد جدید ثبت می شود.

تابع build_domain: مطابق با ورژن های قبلی

تابع build_neighbors: مطابق با ورژن های قبلی

تابع consistent: مطابق با ورژن های قبلی

تابع forward_check

مطابق با ورژن ۲، تنها تفاوت در آپدیت مقدار pruned_states بعد از pruning هست.

```
COUNTERS["PRUNED_STATES"] += 1
```

تابع undo: مطابق با ورژن ۲

تابع select_next_variable: مطابق با ورژن های قبلی

تابع backtrack

مطابق با ورژن ۲، تنها تفاوت در آپدیت مقدار backtrack_calls و max_recursion_depth است.

```
COUNTERS["BACKTRACK_CALLS"] += 1
```

```
COUNTERS["MAX_RECURSION_DEPTH"] = max(COUNTERS["MAX_RECURSION_DEPTH"], depth)
```

تابع solve_3sat_csp: مطابق با ورژن ۲

تابع extract_variable_assignment

```
def extract_variable_assignment(solution):
    final = {}
```

یک دیکشنری خالی برای ذخیره مقادیر نهایی متغیرها می سازد.

```
for clause_state in solution.values():
```

در دیکشنری solution که خروجی الگوریتم است، پیمایش می کند. هر clause_state خودش یک دیکشنری کوچک است مثلاً: {'x1': True, 'x2': False}.

```
final.update(clause_state)
```

این خط کلیدی است. متد update() در پایتون، محتویات دیکشنری clause_state را به دیکشنری final اضافه می کند. اگر متغیری مثل x2 در هر دو وجود داشته باشد، مقدار جدید جایگزین می شود. چون که مسئله سازگار است، مقادیر یکسان هستند.

```
return final
```

در نهایت یک دیکشنری تمیز شامل نام هر متغیر و مقدار True یا False آن برمی گرداند.

تابع run_instance

تابعی برای نمایش بهتر خروجی ها و همچنین استفاده از ابزار time برای اندازه گیری زمان اجرا

ب) آیا با هر بار اجرای الگوریتم انتظار داریم یک جواب منحصر به فرد بدست آید؟

به طور کلی خیر، لزوماً با هر بار اجرای الگوریتم یک جواب منحصر به فرد به دست نمی آید. در مسئله‌ی 3SAT ممکن است چندین انتساب مختلف از مقادیر True و False به متغیرها وجود داشته باشد که همه‌ی گزاره‌ها را ارضا کنند. الگوریتم Backtracking با Forward Checking معمولاً پس از یافتن اولین جواب معتبر متوقف می‌شود، بنابراین بسته به عواملی مانند:

- ترتیب بررسی مقادیر دامنه‌ها
- ترتیب انتخاب متغیرها در صورت برابر بودن اندازه دامنه‌ها
- نحوه‌ی پیمایش دامنه‌ها

ممکن است در اجراهای مختلف جواب‌های متفاوتی به دست آید، در حالی که همه‌ی آن‌ها معتبر هستند. بنابراین انتظار داشتن یک جواب یکتا از اجرای الگوریتم درست نیست مگر اینکه مسئله ذاتاً فقط یک جواب داشته باشد.

تابع نوشته شده در این تمرین، با هر بار تکرار یک جواب منحصر را به دست می آورد.

آیا یک مسئله‌ی SAT-3 لزوماً دارای جواب است؟

خیر، یک مسئله‌ی SAT-3 لزوماً قابل ارضا نیست.

در واقع مسائل SAT به دو دسته تقسیم می‌شوند:

- Satisfiable (SAT): حداقل یک انتساب برای متغیرها وجود دارد که همه‌ی گزاره‌ها را True می‌کند.
- Unsatisfiable (UNSAT): هیچ انتسابی از مقادیر متغیرها وجود ندارد که تمام گزاره‌ها را همزمان ارضا کند.

در مسائل 3SAT ممکن است به دلیل تناقض بین گزاره‌ها هیچ مقداردهی سازگاری وجود نداشته باشد. به عنوان مثال اگر قیود موجود در گزاره‌ها یک متغیر را مجبور کنند که همزمان مقدار True و False داشته باشد، مسئله غیرقابل ارضا خواهد بود.

به همین دلیل الگوریتم‌های مبتنی بر Backtracking تمام فضای جست‌وجو را بررسی می‌کنند و اگر هیچ انتساب معتبری پیدا نشود، نتیجه‌ی عدم وجود جواب (UNSAT) اعلام می‌شود.

د) چه معیاری برای توصیف پیچیدگی فضایی و زمانی لازم برای حل مسئله در قسمت ب می توان در نظر گرفت. با مشخص کردن معیار پیشنهادی خود به طور دقیق، در یک جدول پیچیدگی فضایی و زمانی محاسبه شده را برای سه مسئله ارسالی با یکدیگر مقایسه نمایید.

برای توصیف پیچیدگی زمانی و فضایی الگوریتم Backtracking همراه با Forward Checking، باید معیارهایی انتخاب کنیم که:

- مستقل از سخت افزار باشند
- رفتار واقعی الگوریتم را نشان دهند
- بین سه مسئله قابل مقایسه باشند

🚦 معیارهای پیچیدگی زمانی (Time Complexity Metrics)

۱. Time Complexity Metrics = Backtrack Calls

تعداد دفعاتی که تابع backtrack فراخوانی شده است.

این مقدار متناظر است با:

- تعداد گره های تولید شده در درخت جست و جو
- تعداد تلاش های واقعی برای مقداردهی به متغیرها

این معیار استانداردترین سنجی زمانی در CSP است.

۲. Consistency Checks

تعداد بررسی های سازگاری انجام شده هنگام Forward Checking.

این معیار بیانگر هزینه محاسباتی داخل هر گره است.

۳. Pruned States (FC)

تعداد حالت هایی که توسط Forward Checking حذف شده اند.

نشان دهنده ی میزان pruning و اثر الگوریتم FC در کاهش زمان.

۴. Runtime (seconds)

زمان واقعی اجرا؛ این معیار به سخت افزار وابسته است ولی برای تکمیل گزارش ثبت می شود.

🚩 معیارهای پیچیدگی فضایی (Space Complexity Metrics)

۱. $\text{Space Complexity} = \text{Max Recursion Depth}$

بیشترین عمق درخت جست‌وجو یا طول پشته‌ی بازگشت در طول اجرا

این مقدار برابر است با:

- بیشترین تعداد متغیرهایی که به صورت همزمان مقداردهی شده‌اند
- حداکثر ارتفاع پشته‌ی بازگشت (call stack)

۲. $\text{Max Domain Size (Total)}$

بیشترین مجموع اندازه دامنه‌های متغیرها در یک لحظه از اجرا. این معیار نشان‌دهنده‌ی بیشترین میزان حافظه‌ای است که برای نگهداری CSP نیاز بوده است.

جدول پیچیدگی فضایی و زمانی

	20-literal	50-literal	100-literal
Time Complexity	97	286	470
Space Complexity	91	218	430

تابع `evaluate_clause`

این تابع بررسی می‌کند که یک `clause` با توجه به مقداردهی فعلی متغیرها برقرار است یا نه.

```
def evaluate_clause(clause, assignment):
```

این تابع دو ورودی می‌گیرد:

- `clause`: یک لیست شامل سه لیترال
- `assignment`: لیستی از مقدار متغیرها (`True` یا `False`) برای تمام متغیرهای مسئله

```
    for lit in clause:
```

در این خط روی تمام لیترال‌های موجود در `clause` حلقه زده می‌شود. چون در 3SAT هر `clause` شامل سه لیترال است، این حلقه سه بار اجرا می‌شود.

```
        var = abs(lit) - 1
```

در ماتریس مسئله، هر عدد نشان‌دهنده شماره متغیر است و علامت منفی نشان‌دهنده نقیض آن است. با گرفتن قدر مطلق (`abs`) شماره متغیر به دست می‌آید و با کم کردن ۱ آن را به اندیس مناسب در لیست `assignment` تبدیل می‌کنیم (چون اندیس‌های پایتون از صفر شروع می‌شوند).

```
        if (lit > 0 and assignment[var]) or (lit < 0 and not assignment[var]):
```

در این قسمت بررسی می‌شود که آیا این لیترال با مقداردهی فعلی درست است یا نه. دو حالت داریم:

- اگر لیترال مثبت باشد ($lit > 0$) مقدار متغیر باید `True` باشد.
- اگر لیترال منفی باشد ($lit < 0$) مقدار متغیر باید `False` باشد (چون نقیض آن در `clause` آمده است).

```
    return True
```

اگر حتی یکی از لیترال‌ها درست باشد، کل `clause` برقرار می‌شود (چون بین آنها OR منطقی وجود دارد). بنابراین تابع فوراً مقدار `True` برمی‌گرداند.

تابع num_unsatisfied

این تابع تعداد گزاره‌های ارضا نشده را در یک مقداردهی فعلی حساب می‌کند. این معیار در WalkSAT برای تشخیص میزان بهبود یا بدتر شدن وضعیت پس از flipping یک متغیر استفاده می‌شود.

```
def num_unsatisfied(clauses, assignment):
```

تابع دو ورودی می‌گیرد:

- clauses: لیستی از همه‌ی clause‌های مسئله (هر کدام شامل سه literal)
- assignment: مقداردهی فعلی متغیرها (لیستی از True/False)

```
count = 0
```

متغیر شمارنده که تعداد گزاره‌های برآورده نشده را نگه می‌دارد.

```
for c in clauses:
```

برای بررسی تک تک clause‌های موجود در مسئله یک حلقه زده می‌شود.

```
if not evaluate_clause(c, assignment):  
    count += 1
```

تابع evaluate_clause بررسی می‌کند که clause ارضا شده است یا خیر. اگر ارضا شده نباشد، مقدار آن clause در assignment فعلی False است، پس باید شمارنده یک واحد افزایش یابد.

```
return count
```

در پایان تعداد کل clause‌هایی که هنوز درست نشده‌اند برگردانده می‌شود.

تابع flip

وظیفه این تابع، Flip مقدار منطقی یک متغیر خاص است.

```
def flip(var, assignment):
```

این تابع دو ورودی می‌گیرد:

- var: اندیس متغیری که می‌خواهیم مقدارش را عوض کنیم
- assignment: لیست مقداردهی فعلی به کل متغیرها.

```
assignment[var] = not assignment[var]
```

این خط، مقدار فعلی متغیر در اندیس مورد نظر خوانده می‌شود، با استفاده از عملگر not معکوس می‌گردد و دوباره در همان جایگاه ذخیره می‌شود.

```
def walksat(clauses, n_vars, max_tries, max_flips, p):
```

ورودی‌ها:

- clauses: لیست همه clause‌های مسئله
- n_vars: تعداد متغیرها
- max_tries: حداکثر تعداد شروع‌های تصادفی
- max_flips: حداکثر تعداد تغییر متغیر در هر تلاش
- p: احتمال انتخاب حرکت تصادفی

```
for _ in range(max_tries):
```

الگوریتم چند بار از یک مقداردهی تصادفی جدید شروع می‌کند. این کار کمک می‌کند از گیر افتادن در minimum محلی فرار کنیم.

```
assignment = [random.choice([True, False]) for _ in range(n_vars)]
```

برای هر متغیر مقدار True یا False به صورت تصادفی انتخاب می‌شود.

```
for _ in range(max_flips):
```

در هر تلاش، حداکثر max_flips بار اجازه داریم یک متغیر را تغییر دهیم.

```
unsat_clauses = [c for c in clauses if not evaluate_clause(c,
assignment)]
```

اینجا همه clause‌هایی که در مقداردهی فعلی False هستند پیدا می‌شوند.

```
if not unsat_clauses:
    end_time = time.perf_counter()
    return assignment, (end_time - start_time)
```

اگر هیچ clause نقض نشده باشد یعنی: assignment فعلی یک جواب معتبر است.

```
clause = random.choice(unsat_clauses)
```

یکی از clause‌های نقض شده به صورت تصادفی انتخاب می‌شود.

```
if random.random() < p:
```


با احتمال p یک حرکت کاملاً تصادفی انجام می‌دهیم. این کار کمک می‌کند از local minimum خارج شویم.

```
lit = random.choice(clause)
var = abs(lit) - 1
flip(var, assignment)
```

مراحل حالت تصادفی:

- یک literal از clause انتخاب می‌شود
- شماره متغیر استخراج می‌شود
- مقدار آن متغیر flip می‌شود

```
else:
    best_var = None
    best_score = float("inf")
```

حالت حریصانه

- اگر حرکت تصادفی انجام نشود، الگوریتم تلاش می‌کند بهترین متغیر برای flip را پیدا کند.

```
for lit in clause:
```

چون clause سه literal دارد، حداکثر سه متغیر بررسی می‌شود.

```
var = abs(lit) - 1
flip(var, assignment)
```

به صورت موقت مقدار متغیر تغییر داده می‌شود.

```
score = num_unsatisfied(clauses, assignment)
```

تعداد clause‌های نقض شده بعد از این flip محاسبه می‌شود.

```
flip(var, assignment)
```

چون flip آزمایشی بود، دوباره متغیر را برمی‌گردانیم.

```
if score < best_score:
    best_score = score
    best_var = var
```

متغیری انتخاب می‌شود که کمترین clause نقض شده ایجاد کند.

```
if best_var is None:  
    lit = random.choice(clause)  
    best_var = abs(lit) - 1
```

اگر به هر دلیل متغیر مناسبی پیدا نشود، یک literal تصادفی انتخاب می‌شود.

```
flip(best_var, assignment)
```

در نهایت بهترین متغیر واقعاً تغییر داده می‌شود.

آیا با هر بار اجرای الگوریتم انتظار داریم یک جواب منحصر به فرد بدست آید؟

خیر.

WalkSAT نوعی الگوریتم جستجوی تصادفی است؛ هر بار مقداردهی اولیه متفاوت است و با همان assignment نیز چندین مسیر ممکن تا جواب نهایی وجود دارد. بنابراین اگر مسأله چندین جواب قابل قبول داشته باشد (که برای اکثر مسائل 3SAT همینطور است)، اجرای مجدد الگوریتم معمولاً مقادیر متفاوتی برمی گرداند.

اگر فقط یک جواب وجود داشته باشد، با اجرای مجدد معمولاً همان را برمی گرداند (جز زمانی که Maxtries و Maxflips کافی نباشد، که ممکن است هیچ جوابی پیدا نشود).

مقادیر مناسب برای Maxtries ، Maxflip و p چیست؟

پارامترهای مناسب (p, Maxflips, Maxtries):

- Maxtries: برای ۲۰ متغیر: ۲۰ تا ۵۰، برای ۵۰ تا ۱۰۰، برای ۱۰۰ متغیر: حداقل ۱۰۰-۲۰۰ (یا بیشتر اگر جواب دیر پیدا شود)
- Maxflips: حدود ۵ تا ۱۰ برابر تعداد متغیرها (یعنی برای ۵۰ متغیر: ۲۵۰ تا ۵۰۰، برای ۱۰۰ تا ۵۰۰ یا بیشتر)
- p: مقدار بین ۰/۳ تا ۰/۵۵ مناسب است. اگر به سختی جواب پیدا شود، بهتر است کمی زیادتر شود (ولی معمولاً بالای ۰/۷ خیلی تصادفی و اثر منفی می شود).

آیا این مقادیر مناسب به ماهیت مسئله وابسته هستند؟ به طور دقیق توضیح دهید.

بله، کاملاً وابسته اند.

مسائل خوش خیم (کم قید یا خیلی overconstrained) راحت حل می شوند، حتی با مقادیر کوچک. مسائل نزدیک نقطه بحرانی (که تقریباً نیمی از پاسخ های تصادفی جواب ندارند) معمولاً به تکرار و پارامترهای بزرگتری نیاز دارند .

الگوریتم Clause-Based CSP با Backtracking و Forward Checking در هر سه نمونه‌ی p_20، p_50 و p_100 موفق به یافتن جواب SAT شد. با افزایش اندازه‌ی مسئله، تعداد Backtrack Calls (از ۹۷ به ۴۷۰) و Consistency Checks به‌صورت تدریجی افزایش یافته که نشان‌دهنده‌ی رشد فضای جست‌وجو است. با این حال، رشد این مقادیر کنترل‌شده باقی مانده است.

افزایش تعداد Pruned States بیانگر اثربخشی Forward Checking در حذف زودهنگام حالت‌های ناسازگار است. همچنین، Max Recursion Depth و Max Total Domain Size تقریباً متناسب با اندازه‌ی مسئله رشد کرده و پیچیدگی فضایی الگوریتم را نشان می‌دهند. زمان اجرای کل نیز با افزایش اندازه‌ی مسئله افزایش یافته، اما همچنان در بازه‌ی بسیار کوچک باقی مانده است.

در مجموع، نتایج نشان می‌دهد که این روش برای حل مسائل 3SAT در ابعاد متوسط کارا است، اما برای مسائل بزرگ‌تر نیاز به الگوریتم‌های پیشرفته‌تر مانند CDCL وجود دارد.

Clause based CSP with BT-FC (with Trail Stack)

```
Solving instance: p_20
Result: SAT (Solution found)
--- Complexity Metrics ---
Backtrack Calls(Time Complexity): 97
Consistency Checks:          5015
Pruned States (FC):          721
Max Recursion Depth(Space Complexit): 91
Max Domain Size (total): 637
Runtime (seconds):           0.007444
```

```
Solving instance: p_50
Result: SAT (Solution found)
--- Complexity Metrics ---
Backtrack Calls(Time Complexity): 286
Consistency Checks:          15854
Pruned States (FC):          2175
Max Recursion Depth(Space Complexit): 218
Max Domain Size (total): 1526
Runtime (seconds):           0.039217
```

```
Solving instance: p_100
Result: SAT (Solution found)
--- Complexity Metrics ---
Backtrack Calls(Time Complexity): 470
Consistency Checks:          24563
Pruned States (FC):          3308
Max Recursion Depth(Space Complexit): 430
Max Domain Size (total): 3010
Runtime (seconds):           0.130035
```

نتایج اجرای الگوریتم WalkSAT نشان می‌دهد که با افزایش تعداد متغیرها، زمان اجرای الگوریتم نیز به طور قابل توجهی افزایش می‌یابد. برای مثال، در مسئله با ۲۰ متغیر زمان اجرا حدود 0.006 ثانیه، برای ۵۰ متغیر حدود 0.33 ثانیه و برای ۱۰۰ متغیر حدود 3.75 ثانیه به دست آمده است که نشان‌دهنده افزایش هزینه جستجو با بزرگ‌تر شدن فضای مسئله است. همچنین باید توجه داشت که WalkSAT یک الگوریتم تصادفی (stochastic) مبتنی بر جستجوی محلی است؛ بنابراین در هر بار اجرای الگوریتم ممکن است مقداردهی نهایی متغیرها و حتی زمان اجرا متفاوت باشد. این تفاوت به دلیل مقداردهی اولیه تصادفی و انتخاب تصادفی متغیرها هنگام عملیات flip رخ می‌دهد. در نتیجه، ممکن است یک اجرای الگوریتم سریع‌تر به جواب برسد یا جواب متفاوتی نسبت به اجرای دیگر تولید کند، در حالی که هر دو پاسخ می‌توانند قیود مسئله را ارضا کنند.

```
walksat(clauses= p_20, n_vars=20, max_tries=20, max_flips=200, p= 0.5)
([True,
  True,
  False,
  ....
  False,
  True,
  True],
Run time: 0.0061506000347435474)
```

```
walksat(clauses= p_50, n_vars=50, max_tries=50, max_flips=500, p= 0.5)
([False,
  False,
  True,
  ....
  True,
  True,
  False],
Run time 0.33890660002361983)
```

```
walksat(clauses= p_100, n_vars=100, max_tries=100, max_flips=1000, p=0.5)
([True,
  True,
  True,
  ....
  False,
  False,
  True],
Run time 3.758742699981667)
```

این تمرین به بررسی و پیاده‌سازی دو رویکرد متفاوت برای حل مسئله 3SAT، شامل روش‌های Backtracking با Forward Checking و WalkSAT پرداخت.

در بخش اول، پیاده‌سازی الگوریتم Backtracking با Forward Checking بر پایه مدل بندمحور (Clause-centric) انجام شد که سازگاری بیشتری با ساختار 3SAT از خود نشان داد. بهینه‌سازی مدیریت حالت با استفاده از Trail Stack به جای deepcopy، بهبود چشمگیری در کارایی زمانی و حافظه‌ای الگوریتم به همراه داشت و آن را برای مسائل در ابعاد متوسط کارآمدتر ساخت. استفاده از هیوریستیک‌های MRV (Minimum Remaining Values) برای انتخاب فضای جستجو را به طرز محسوسی کاهش داد. معیارهای ارزیابی شامل تعداد Backtrack Calls و Max Recursion Depth به وضوح رشد نمایی پیچیدگی را با افزایش تعداد متغیرها نشان دادند، که تاییدی بر ماهیت NP-Complete مسئله 3SAT است. در مجموع، رویکرد بندمحور با Trail Stack و هیوریستیک‌های ذکر شده، راه‌حلی پایه‌ای و مناسب برای حل مسائل 3SAT در ابعاد متوسط فراهم آورد و مسیر را برای روش‌های پیشرفته‌تر مانند CDCL (Conflict-Driven Clause Learning) و الگوریتم‌های مبتنی بر یادگیری ماشین هموار ساخت.

در بخش دوم، الگوریتم WalkSAT به عنوان یک رویکرد جستجوی محلی تصادفی برای حل 3SAT مورد بررسی قرار گرفت. نتایج نشان داد که WalkSAT به دلیل ماهیت اکتشافی خود و توانایی در گریز از کمینه‌های محلی از طریق انتخاب‌های تصادفی (با احتمال p)، قادر است به یک جواب ارضاکنده مسئله دست یابد. با این حال، ماهیت تصادفی مقداردهی اولیه و همچنین فرآیند flipping متغیرها، منجر به این می‌شود که اجراهای مختلف الگوریتم ممکن است به جواب‌های متفاوتی منجر شوند، به خصوص در مسائلی با چندین حل ممکن. عملکرد WalkSAT به شدت به تنظیم دقیق پارامترهای Maxtries (تعداد تلاش‌ها)، Maxflips (حداکثر تغییرات در هر تلاش) و احتمال p وابسته است. افزایش Maxtries و Maxflips اغلب احتمال یافتن جواب را بالا می‌برد اما به افزایش زمان اجرا نیز منجر می‌شود. پارامتر p نیز نقش حیاتی در ایجاد تعادل بین اکتشاف تصادفی و بهره‌برداری از بهترین حرکت greedy ایفا می‌کند؛ مقادیر متوسط برای p معمولاً بهینه‌ترین عملکرد را ارائه می‌دهند. در نهایت، مشخص شد که انتخاب بهینه این پارامترها کاملاً به ساختار و سختی هر نمونه مسئله وابسته است و نیاز به آزمایش دقیق بر اساس ویژگی‌های خاص مسئله دارد.